



MEKELLE UNIVERSITY

MEKELLE INSTITUTE OF TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY



# *Advanced Database Systems*

## *Module*

*All Chapters in One  
Compiled Module*

*Compiled By: Mezgebe Mehari | To: 2<sup>nd</sup> Year IT students  
3<sup>rd</sup> Year IT students*

*MU-MIT, Ethiopia  
Semester-II, 2011 E.C*

## ***Table of Contents***

|                                                                          |           |
|--------------------------------------------------------------------------|-----------|
| <b>CHAPTER - ONE: QUERY PROCESSING AND OPTIMIZATION.....</b>             | <b>3</b>  |
| 1.1 Overview of Query Optimization.....                                  | 3         |
| 1.2 Translating SQL Queries into Relational Algebra.....                 | 4         |
| 1.3 Basic Algorithms for Executing Query Operations.....                 | 6         |
| 1.3.1 External Sorting.....                                              | 6         |
| 1.3.2 Implementing the SELECT Operation .....                            | 7         |
| 1.3.3 Implementing the JOIN Operation .....                              | 9         |
| 1.4 Using Heuristics in Query Optimization.....                          | 11        |
| 1.4.1 Notation for Query Trees and Query Graphs .....                    | 11        |
| 1.5 Using Selectivity and Cost Estimates in Query Optimization.....      | 12        |
| 1.5.1 Catalog Information Used in Cost Functions.....                    | 13        |
| 1.6 Overview of Query Optimization in ORACLE.....                        | 14        |
| 1.7 Semantic Query Optimization.....                                     | 16        |
| <b>CHAPTER - TWO: TRANSACTION PROCESSING.....</b>                        | <b>17</b> |
| 2.1 Introduction, Transaction Processing and System Concepts.....        | 17        |
| 2.2 Desirable Properties of Transactions.....                            | 17        |
| 2.3 Schedules Based on Recoverability and Serializability .....          | 17        |
| 2.4 Transaction Support in SQL .....                                     | 17        |
| <b>CHAPTER - THREE: CONCURRENCY CONTROL TECHNIQUES.....</b>              | <b>17</b> |
| 3.1 Introduction, Transaction Processing and System Concepts.....        | 17        |
| 3.2 Desirable Properties of Transactions.....                            | 17        |
| 3.3 Schedules Based on Recoverability and Serializability .....          | 17        |
| 3.4 Transaction Support in SQL .....                                     | 17        |
| <b>CHAPTER - FOUR: TRANSACTION PROCESSING.....</b>                       | <b>18</b> |
| 4.1 Locking Techniques for Concurrency Control.....                      | 18        |
| 4.2 Concurrency Control Based on Timestamp Ordering.....                 | 18        |
| 4.3 Multiversion Concurrency Control Techniques.....                     | 18        |
| 4.4 Validation (Optimistic) Concurrency Control Techniques .....         | 18        |
| 4.5 Granularity of Data Items and Multiple Granularity Locking .....     | 18        |
| 4.6 Using Locks for Concurrency Control in Indexes .....                 | 18        |
| <b>CHAPTER - FIVE: DISTRIBUTED DATABASE SYSTEM.....</b>                  | <b>18</b> |
| 5.1 Distributed Database Concepts, Data Fragmentation, Replication ..... | 18        |

|     |                                                                  |    |
|-----|------------------------------------------------------------------|----|
| 5.2 | Allocation Techniques for Distributed Database Design .....      | 18 |
| 5.3 | Concurrency Control and Recovery in Distributed Databases .....  | 18 |
| 5.4 | Transaction Management in Distributed Databases.....             | 19 |
| 5.5 | Types of Distributed Databases and Distributed DB in Oracle..... | 19 |

---

## CHAPTER - ONE: QUERY PROCESSING AND OPTIMIZATION

---

### Query Processing and Optimization

---

#### 1.1 Overview of Query Optimization

---

- ♣ In this chapter we will discuss the *techniques* used by a **DBMS** to process, *optimize*, and *execute* high-level queries.
- ♣ A query expressed in a high-level query language such as SQL must first be *scanned, parsed, and validated*. The **scanner** identifies the language tokens—such as SQL keywords, attribute names, and relation names—in the text of the query, whereas the **parser** checks the query syntax to determine whether it is formulated according to the *syntax rules* (rules of grammar) of the query language.
- ♣ The query must also be **validated**, by checking that all attribute and relation names are valid and semantically meaningful names in the schema of the particular database being queried.
- ♣ An internal representation of the query is then created, usually as a tree *data structure* called a **query tree**. It is also possible to represent the query *using a graph* data structure called a **query graph**.
- ♣ The DBMS must then devise an **execution strategy** for retrieving the result of the query from the database files. A query typically has many possible execution strategies, and *the process of choosing a suitable* one for processing a query is known as **query optimization**.
- ♣ From **fundamentals of database** course of chapter 2.5, the *runtime database processor* has the task of running the query code, whether in **compiled** or **interpreted** mode, to produce the query result. If a *runtime error results*, an error message is generated by the *runtime database processor*.
- ♣ The term *optimization* is actually a **misnomer** because in some cases the chosen execution plan is not the **optimal** (best) strategy—it is just a *reasonably efficient strategy* for executing the query. Finding the optimal strategy is usually too **time-consuming except** for the simplest of queries and may require information on how the files are implemented and even on the contents of the files—information that may not be fully available in the DBMS catalog.
- ♣ Hence, *planning of an execution strategy* may be a more **accurate** description than *query optimization*.
- ♣ If a DBMS provides only a navigational language, there is *limited need or opportunity* for extensive query optimization by the DBMS; instead, the programmer is given the capability to choose the "**optimal**" execution strategy. *On the other hand*, a high-level query language—such as SQL for relational DBMSs (RDBMSs) or OQL for object DBMSs (ODBMSs)—is *more declarative in nature* because it specifies what the intended results of the query are, rather than identifying the details of *how* the result

should be obtained. Query optimization is thus necessary for queries that are specified in a *high-level query* language.

- ♣ A relational DBMS must *systematically* evaluate alternative query execution strategies and choose a *reasonably efficient or optimal* strategy. Each DBMS typically has a number of general database access algorithms that implement relational operations such as SELECT or JOIN or combinations of these operations. **Only execution strategies** that can be implemented by the DBMS access algorithms and that apply to the particular query and particular physical database design can be considered by the query optimization module.
- ♣ We will start in Section 1.2 with a general discussion *of how SQL queries are typically translated into relational algebra* queries and then optimized. We will then discuss algorithms for implementing relational operations in Section 1.3. Following this, we give an overview of query optimization strategies. There are *two main techniques for implementing query optimization*.
  - The first technique is based on **heuristic rules** for **ordering** the operations in a query execution strategy. A heuristic is a rule that works well in most cases but is *not guaranteed to work well in every* possible case. The rules typically reorder the operations in a query tree.
  - The second technique involves **systematically estimating** the cost of different execution strategies and choosing the execution plan with the *lowest cost estimate*.
- ♣ The **two techniques** are usually **combined** in a query **optimizer**. We will discuss heuristic optimization in Section 1.2 in detail. We then provide a brief overview of the factors considered during query optimization in the ORACLE commercial RDBMS in Section 1.6. Section 1.7 introduces the topic of **semantic query optimization**, in which **known constraints** are used to devise efficient query execution strategies.

## 1.2 Translating SQL Queries into Relational Algebra

---

- ♣ In practice, *SQL is the query language* that is used in *most commercial RDBMSs*. An SQL query is **first** *translated into an equivalent extended relational algebra expression* – represented as a *query tree data structure* – that is then optimized.
- ♣ **Typically**, SQL queries are decomposed *into query blocks*, which form the basic units that can be translated into the algebraic operators and optimized. A query block contains a single *SELECT-FROM-WHERE expression*, as well as *GROUP BY and HAVING* clauses if these are part of the block.
  - Hence, *nested queries within a query* are identified as separate query blocks. **Because** SQL includes *aggregate operators* – such as MAX, MIN, SUM, AND COUNT – these operators must also be included in the

extended algebra, as we discussed in Fundamentals of Database Systems.

- ♣ Consider the **following SQL query** on the **EMPLOYEE** relation:

```
SELECT LNAME, FNAME
FROM EMPLOYEE
WHERE SALARY > (SELECT MAX
                 (SALARY) FROM
                 EMPLOYEE WHERE
                 DNO=5);
```

- ♣ This query includes a **nested subquery** and hence would be decomposed *into two blocks*.

- The *inner block* is:

```
(SELECT MAX (SALARY)
FROM EMPLOYEE
WHERE DNO=5)
```

- And the *outer block* is:

```
SELECT LNAME, FNAME FROM
EMPLOYEE WHERE SALARY > C
```

- Where *c represents* the result returned from the inner block.
- The inner block could be translated into the extended relational algebra expression:

$$\text{MAX SALARY}(\text{S}_{\text{DNO}=5}(\text{EMPLOYEE}))$$

- And the *outer block* into the expression:

$$\text{p}_{\text{LNAME, FNAME}}(\text{S}_{\text{SALARY}>\text{C}}(\text{EMPLOYEE}))$$

- ♣ The **query optimizer** would then choose an *execution plan for each block*. We should note that in the above example, the *inner block needs* to be evaluated **only once** to produce the *maximum salary*, which is then used—*as the constant c*—by the **outer block**.
  - We normally call such kind of nested query as an *uncorrelated nested query*.
  - It is much harder to *optimize the more complex correlated nested queries*, where a *tuple variable from the outer block* appears in the **WHERE-clause** of the inner block.

## 1.3 Basic Algorithms for Executing Query Operations

- ♣ An **RDBMS** must include **algorithms** for implementing the different types of relational operations (*as well as other types of operations*) that can appear in a query *execution strategy*.
  - These operations include the basic and extended relational algebra operations discussed in FDBS and, in many cases, combinations of these operations. For each such operation or combination of operations, one or more algorithms would typically be available to execute the operation(s).
- ♣ An **algorithm** may apply only to *particular storage structures* and *access paths*; if so, it can only be used if the files involved in the operation include these access paths.
  - In this section we discuss typical algorithms used to implement **SELECT**, **JOIN**, and other *relational* operations. We begin by discussing *external sorting*, which is at the *heart of many relational* operations that utilize *sort-merge strategies*. Then we discuss algorithms for implementing the SELECT and JOIN operation.

### 1.3.1 External Sorting

- ♣ Sorting is one of the **primary algorithms** used in *query processing*.
  - For example, whenever an SQL query specifies an ORDER BY-clause, the query result must be sorted.
- ♣ Sorting is *also a key component in sort-merge algorithms* used for **JOIN** and *other* operations (such as **UNION** and **INTERSECTION**), and in duplicate elimination algorithms for the PROJECT operation (when an SQL query specifies the DISTINCT option in the SELECT clause).
- ♣ **External sorting** refers to sorting algorithms that are suitable for large files of records stored on disk that do not fit entirely in *main memory*, such as *most database* files.
  - The typical external sorting algorithm uses a **sort-merge strategy**, which starts by sorting small **sub-files**—called **runs**—of the main file and then *merges the sorted* runs, creating larger sorted sub-files that are merged in turn.
- ♣ The **sort-merge algorithm**, like other database algorithms, requires *buffer space* in main memory, where the *actual sorting* and *merging* of the runs is performed.
  - **It consists of two phases:**
    1. The **sorting** phase and
    2. The **merging** phase
- ♣ In the *sorting phase*, runs (portions or pieces) of the file that can fit in the available buffer space are read into *main memory*, sorted using an *internal sorting*



*algorithm*, and written back to disk as temporary sorted subfiles (or runs). The size of a run and *number of initial runs* is dictated by the *number of file blocks (b)* and the *available buffer space*.

- For example, if  $r = 5$  blocks and the size of the file  $b = 1024$  blocks, then  $r = 205$  initial runs each of size 5 blocks (except the last run which will have 4 blocks). Hence, after the sort phase, 205 sorted runs are stored as temporary sub-files on disk.
- ♣ In the *merging phase*, the sorted runs are merged during one or more *passes*. The *degree of merging* is the *number of runs* that can be *merged* together in *each* pass. In each pass, one buffer block is needed to hold one block from each of the runs being *merged*, and one *block* is needed for containing *one block* of the merge result.
  - In our example,  $r = 4$  (*four-way merging*), so the 205 initial sorted runs would be merged into 52 at the end of the first pass, which are then merged into 13, then 4, then 1 run, which means that *four passes* are needed. The minimum of 2 gives the *worst-case performance* of the algorithm, which is:
    - $(2 * b) + (2 * (b * (\log_2 b)))$
- ♣ The *first term represents* the number of block *accesses* for the *sort* phase, since each file block is *accessed twice*—*once* for *reading* into memory and *once* for *writing* the records back to disk after sorting.
- ♣ The *second term represents* the number of block *accesses* for the *merge* phase, assuming the *worst-case* of 2. In general, the *log is taken to the base* and the expression for number of block accesses becomes.

### 1.3.2 Implementing the SELECT Operation

- ♣ There are many options for executing a *SELECT* operation; some depend on the file having specific access paths and may apply only to certain types of selection conditions.
- ♣ We discuss some of the *algorithms for implementing SELECT* in this section. We will use the following operations, specified on the relational database, to *illustrate* our discussion:

(OP1):  $S_{SSN='123456789'}(EMPLOYEE)$   
 (OP2):  $S_{DNUMBER>5}(DEPARTMENT)$   
 (OP3):  $S_{DNO=5}(EMPLOYEE)$   
 (OP4):  $S_{DNO=5 \text{ AND } SALARY>30000 \text{ AND } SEX='F'}(EMPLOYEE)$   
 (OP5):  $S_{ESSN='123456789' \text{ AND } PNO=10}(WORKS\_ON)$



## Search Methods for Simple Selection

- ♣ A number of *search algorithms are possible* for selecting records from a file. These are also known as **file scans**, because they scan the records of a file to search for and retrieve records that satisfy a selection condition. If the search algorithm involves the use of an index, the index search is called an **index scan**.
- ♣ The following search methods (S1 through S6) are examples of some of the search algorithms that can be used to implement a select operation:

### S1. *Linear search (brute force):*

Retrieve *every record* in the file, and test whether its attribute values satisfy the selection condition.

### S2. *Binary search:*

If the selection condition involves an *equality comparison* on a *key attribute* on which the file is ordered, binary search—which is more efficient than linear search—can be used. An example is OP1 if SSN is the ordering attribute for the EMPLOYEE file.

### S3. *Using a primary index (or hash key):*

If the selection condition involves an *equality comparison* on a *key attribute* with a *primary index* (or *hash key*)—for example, SSN = '123456789' in OP1—use the primary index (or hash key) to retrieve the record. Note that this condition retrieves a single record (at most).

### S4. *Using a primary index to retrieve multiple records:*

If the comparison condition is  $>$ ,  $\geq$ ,  $<$ , or  $\leq$  on a key field with a *primary index*—for example, DNUMBER  $>$  5 in OP2—use the index to find the record satisfying the corresponding equality condition (DNUMBER = 5), then retrieve all subsequent records in the (ordered) file. For the condition DNUMBER  $<$  5, retrieve all the preceding records.

### S5. *Using a clustering index to retrieve multiple records:*

If the selection condition involves an equality comparison on a *non-key attribute* with a clustering index—for example, DNO = 5 in OP3—use the index to retrieve all the records satisfying the condition.

**S6. Using a secondary (-tree) index on an equality comparison:**

This search method can be used to retrieve a single record if the indexing field is a **key** (has unique values) or to retrieve multiple records if the indexing field is **not a key**. This can also be used for **comparisons** involving **>**, **>=**, **<**, or **<=**.

- ♣ Method **S1** applies to **any file**, but all the **other methods depend on having the appropriate access path on the attribute** used in the selection condition. Methods **S4** and **S6** can be used to retrieve records in a certain **range**—for example, **30000<=SALARY<=35000**. Queries involving such conditions are called **range queries**.

### 1.3.3 Implementing the JOIN Operation

- ♣ The **JOIN** operation is one of the most time-consuming operations in query processing. Many of the join operations encountered in queries are of the **EQUIJOIN** and **NATURAL JOIN** varieties, so we consider only these two here. For the remainder of this chapter, the term **join** refers to an **EQUIJOIN** (or **NATURAL JOIN**).
- ♣ There are many possible ways to implement a **two-way join**, which is a join on two files. Joins involving **more than two** files are called **multiway joins**. The number of possible ways to execute multiway joins grows very rapidly. In this section we discuss techniques for implementing only two-way joins.
- ♣ The algorithms we consider are for join operations of the form:

$$R \bowtie A=B \ S$$

where **A** and **B** are **domain-compatible attributes** of **R** and **S**, **respectively**. The methods we discuss can be extended to more general forms of join.

- We illustrate **four** of the **most common techniques** for performing such a join, using the **following example** operations:

(OP6): **EMPLOYEE**DNO=DNUMBER **DEPARTMENT**

(OP7): **DEPARTMENT**MGRSSN=SSN **EMPLOYEE**

## Methods for Implementing Joins

### J1. Nested-loop join (brute force):

For each record **t** in **R** (**outer** loop), retrieve every record **s** from **S** (**inner** loop) and test **whether** the two records **satisfy** the **join** condition **t[A] = s[B]**.

- J2. **Single-loop join (using an access structure to retrieve the matching records):**

If an *index* (or *hash* key) exists for *one* of the *two join* attributes—say, *B* of *S*—retrieve *each record* *t* in *R*, one at a time (*single* loop), and *then use the access* structure to retrieve directly *all matching records* *s* from *S* that satisfy  $s[B] = t[A]$ .

### J3. Sort-merge join:

- If the records of *R* and *S* are *physically sorted* (ordered) by value of the join attributes *A* and *B*, respectively, we can implement the join in the *most efficient* way possible.
- *Both files are scanned concurrently* in order of the join attributes, *matching* the records that have the *same values* for *A* and *B*. If the files are *not sorted*, they may be *sorted first* by using *external* sorting.
  - In this method, *pairs of file blocks* are copied into *memory buffers* in order and the records of each file are scanned only once each for matching with the other file—*unless both* *A* and *B* are *non-key* attributes, in which case the method needs to be modified slightly.
- The *indexes* provide the ability to access (scan) the records in order of the join attributes, but the records themselves are *physically scattered* all over the *file blocks*, so this method may be quite inefficient, as every record access may involve accessing a different *disk block*.

### J4. Hash-join:

- The *records of files R and S are both hashed* to the *same hash file*, using the same *hashing function* on the join attributes *A* of *R* and *B* of *S* as *hash keys*.
- *First*, a *single pass* through the *file with fewer records* (say, *R*) hashes its records to the *hash* file *buckets*;
  - This is called the *partitioning phase*, since the records of *R* are *partitioned* into the *hash buckets*.
- In the *second phase*, called the *probing phase*, a single pass through the other file (*S*) *then hashes* each of its *records* to *probe* the appropriate *bucket*, and that record is *combined* with *all matching records* from *R* in *that bucket*.
- This simplified description of hash-join assumes that the smaller of the two files *fits entirely into memory buckets* after the first phase. We will discuss variations of hash-join that do not require this assumption below.

## 1.4 Using Heuristics in Query Optimization

- ♣ In this section we will discuss *optimization techniques* that apply *heuristic rules* to modify the *internal representation* of a *query* – which is usually in the form of a *query tree* or a *query graph* data structure – to improve its expected performance.
- ♣ The *parser of a high-level query* *first* generates an *initial internal representation*, which is then *optimized* according to *heuristic rules*.
  - ♣ Following that, a *query execution plan* is generated to execute groups of operations *based on the access paths available* on the files involved in the query.
- ♣ One of the main *heuristic rules* is to apply *SELECT* and *PROJECT* operations *before* applying the *JOIN* or other binary operations.
  - ♣ This is *because the size* of the *file resulting* from a *binary* operation – such as *JOIN* – is usually a *multiplicative function* of the *sizes* of the *input files*.
  - ♣ The *SELECT* and *PROJECT* operations *reduce the size* of a file and *hence* should be applied *before* a *join* or *other binary* operation.
- ♣ *Query tree* and *query graph* notations can be used *as the basis for the data structures* that are used for *internal representation of queries*.
  - ♣ A *query tree* is used to represent a *relational algebra* or extended relational algebra expression, whereas a *query graph* is used to represent a *relational calculus* expression.

### 1.4.1 Notation for Query Trees and Query Graphs

- ♣ A *query tree* is a *tree data* structure that corresponds to a *relational algebra* expression. It represents the input relations of the query as *leaf nodes* of the tree, and represents the *relational algebra operations* as *internal nodes*.
- ♣ An execution of the *query tree* consists of executing an *internal node* operation whenever its operands are available and then *replacing* that *internal node* by the *relation* that results from *executing* the operation.
  - The *execution terminates* when the *root node* is *executed* and produces the *result relation* for the query.

Example:  $\text{DNUM} = \text{DNUMBER}(\text{DEPARTMENT}) \text{ MGRSSN} = \text{SSN}(\text{EMPLOYEE})$

Corresponds to this query:

```
SELECT P.PNUMBER, P.DNUM, E.LNAME, E.ADDRESS, BDATE
FROM PROJECT AS P, DEPARTMENT AS D, EMPLOYEE AS E
WHERE P.DNUM=D.DNUMBER AND D.MGRSSN=E.SSN AND
PLOCATION='MIT';
```

- ♣ The three *relations* **PROJECT**, **DEPARTMENT**, and **EMPLOYEE** are represented by leaf nodes **P**, **D**, and **E**, while the *relational algebra* operations of the expression are represented by *internal tree nodes*.
- ♣ As we can observe, the *query tree* represents a **specific order** of operations for *executing* a query. A **more neutral** representation of a query is the **query graph** notation.
  - *Relations* in the query are represented by *relation nodes*, which are displayed as *single circles*.
  - *Constant values*, typically from the query *selection* conditions, are represented by *constant nodes*, which are displayed as *double circles*.
  - *Selection* and *join* conditions are represented by the *graph edges*.
  - *Finally*, the attributes to be retrieved from *each relation* are displayed in *square brackets* above each relation.
- ♣ *The query graph* representation *does not* indicate *an order* on which operations to perform *first*. There is *only* a *single graph* corresponding to *each query*.
- ♣ *Although some* optimization *techniques* were *based on query graphs*, it is now generally *accepted* that *query trees* are *preferable* because, in *practice*, the query optimizer needs to show the *order of operations* for query execution, which *is not possible in query graphs*.

## 1.5 Using Selectivity and Cost Estimates in Query Optimization

---

- ♣ A *query optimizer* should *not depend solely* on *heuristic rules*; it should *also estimate* and *compare* the *costs* of *executing* a query using different execution strategies and *should choose* the strategy with the *lowest cost estimate*.
  - For this approach to work, accurate *cost estimates* are required so that different strategies are compared *fairly* and *realistically*.
- ♣ In addition, we must *limit the number of execution strategies* to be considered; otherwise, too much time will be spent making *cost estimates* for the many possible execution strategies.
- ♣ Hence, this approach is *more suitable* for *compiled queries* where the optimization is *done* at *compile time* and the resulting execution strategy code is stored and executed *directly* at *runtime*.
- ♣ For *interpreted queries*, where the *entire process* occurs at *runtime*, a full-scale optimization may slow down the response time.
  - A *more elaborate* optimization is indicated for *compiled queries*, *whereas* a *partial, less time-consuming* optimization works best for *interpreted queries*.
  - We call this approach **cost-based query optimization**, and it *uses traditional optimization techniques* that search the *solution space* to a problem for a solution that minimizes an *objective (cost) function*.



- The *cost functions* used in query optimization are estimates and *not exact cost* functions, so the optimization may *select* a *query* execution strategy that is *not* the *optimal* one.

□ The cost of *executing a query* includes the following components.

**1) Access cost to secondary storage:**

- This is the *cost of searching* for, *reading*, and *writing* data blocks that *reside on secondary storage*, mainly on *disk*.
- The cost of *searching for records* in a file depends on the *type of access structures* on that file, such as *ordering*, *hashing*, and *primary or secondary* indexes.
- In addition, *factors* such as whether the *file blocks* are allocated *contiguously* on the same disk *cylinder* or scattered on the disk affect the *access* cost.

**2) Storage cost:**

- This is the *cost of storing* any *intermediate files* that are generated by *an execution strategy* for the query.

**3) Computation cost :**

- This is the *cost* of performing *in-memory operations* on the data *buffers during query execution*.
- Such operations include *searching* for and *sorting records*, *merging records* for a join, and *performing computations* on field values.

**4) Memory usage cost:**

- This is the *cost pertaining* to the *number of memory buffers needed* during *query execution*.

**5) Communication cost:**

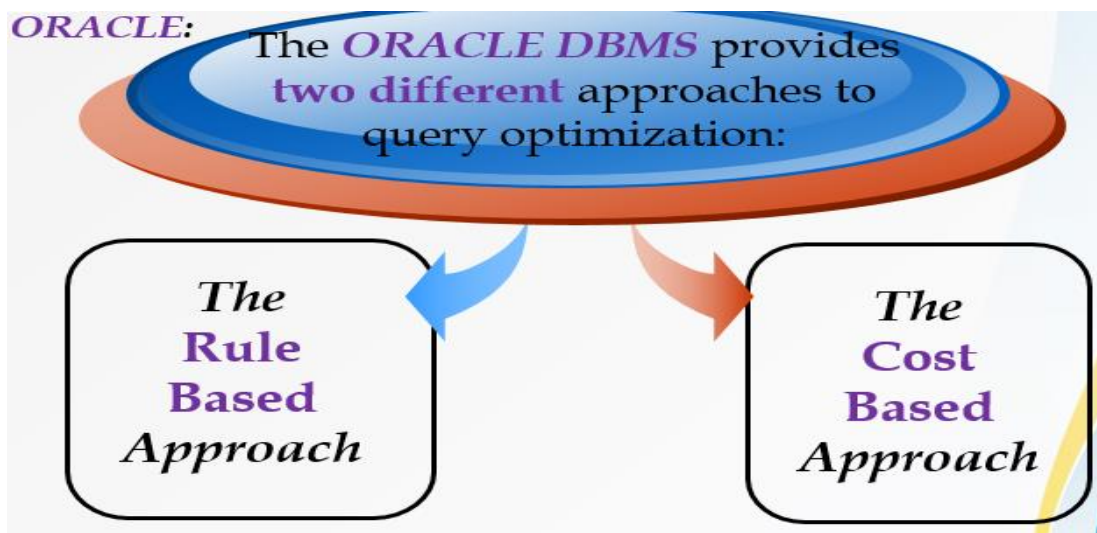
- This is the *cost of shipping the query* and *its results from the database site* to the *site or terminal* where the *query originated*.

### 1.5.1 Catalog Information Used in Cost Functions

- ♣ To *estimate* the *costs* of *various execution strategies*, we must keep track of *any information* that is needed for the *cost functions*.
- ♣ This *information* may be stored in the *DBMS catalog*, where it *is accessed* by the *query optimizer*.
- ♣ **First**, we must know the *size of each file*.
  - For a file whose records are all of the *same type*, the *number of records (tuples) (r)*, the (average) *record size (R)*, and the *number of blocks (b)* (or close estimates of them) are needed.
  - The *blocking factor (bfr)* for the file may also be needed.

- ♣ We must *also keep track* of the *primary access method* and the *primary access attributes* for each file.
  - The *file records* may be *unordered*, *ordered* by an attribute with or without a *primary* or *clustering* index, or *hashed* on a *key* attribute.
  - Information is kept on *all secondary indexes* and *indexing attributes*.
  - The *number of levels (x)* of each *multilevel index* (*primary*, *secondary*, or *clustering*) is needed for *cost functions* that estimate the number of *block accesses* that occur *during query* execution.
  - In some *cost* functions the *number of first-level index blocks* is needed.
- ♣ Another important parameter is the *number of distinct values (d)* of an attribute and its *selectivity (sl)*, which is the *fraction* of records satisfying an *equality condition* on the attribute.
  - This allows estimation of the *selection cardinality* ( $s = sl * r$ ) of an attribute, which is the *average* number of records that will *satisfy an equality selection condition* on that attribute.
    - For a *key attribute*,  $d = r$ ,  $sl = 1/r$  and hence  $s = sl * r$ , since  $sl = 1/r$  then  $s = 1/r * r = 1$ .
    - For a *non-key attribute*, by making an assumption that the  $d$  distinct values are *uniformly distributed* among the records, we *estimate*  $sl = (1/d)$  and hence  $s = sl * r$ , since  $sl = 1/d$  then  $s = 1/d * r = (r/d)$ .
- ♣ **Information** such as the *number of index levels* is *easy* to maintain because it *does not change very* often. **However**, other information may *change frequently*;
  - For example, the *number of records r* in a file changes *every time* a record is *inserted* or *deleted*.
- ♣ The query optimizer will need reasonably close but not necessarily completely *up-to-the-minute values of these parameters* for use in estimating the *cost of various execution strategies*.

## 1.6 Overview of Query Optimization in ORACLE





- With the *rule-based approach*, the **optimizer** chooses *execution plans* based on *heuristically ranked operations*.
- ORACLE maintains *a table of ranked access paths*, where a *lower ranking implies* a **more efficient** approach.
  - ❑ The *access paths range* from table access by **ROWID** (*most efficient*)—where **ROWID** specifies the *record's physical address* that includes:
    1. The **data file**,
    2. **Data block**, and
    3. **Row offset**within the block— to a *full table scan* (least efficient)—*where all rows in the table are searched* by doing **multi-block** reads.
- However, the *rule-based approach* is *being phased out* in **favor** of the *cost-based approach*, where the *optimizer examines alternative access paths and operator algorithms* and chooses the execution plan with *lowest estimated cost*.
- *The catalog tables* containing *statistical information* are used in a *similar fashion*, as described in Section 1.5.
- The **estimated query cost** is *proportional* to the *expected elapsed time* needed to *execute the query* with the *given execution plan*.
  - ❑ The **ORACLE optimizer calculates** this **cost based on the estimated usage of resources**, such as:
    - ✓ **Input/Output**,
    - ✓ **CPU time** and
    - ✓ **Memory Needed**
- The goal of *cost-based optimization* in **ORACLE** is to *minimize the elapsed time* to process the entire query.
- The *cost-based optimization* of ORACLE-8 is a good example of the *sophisticated approach* taken to *optimize SQL queries in commercial RDBMSs*.

## 1.7 Semantic Query Optimization

- A *different approach* to query optimization, called **semantic query optimization**, has been *suggested*.
- This **technique**, which may be used in *combination with the techniques discussed previously*, *uses constraints specified on the database schema* – such as:

- ❑ *Unique attributes* and

- ❑ *Other more complex constraints* – in order to *modify one query into another query* that is more efficient to execute.

- We will not discuss this approach in detail *but only illustrate it with a simple example*. Consider the *SQL query*:

```
SELECT  E.LNAME, M.LNAME
FROM    EMPLOYEE AS E, EMPLOYEE AS M
WHERE   E.SUPERSSN=M.SSN AND
        E.SALARY.M.SALARY
```

- This query **retrieves** the names of *employees who earn more than their supervisors*.
  - Suppose that *we had a constraint on the database schema* that stated that **no employee can earn more than his or her direct supervisor**.
- If the semantic query *optimizer checks for the existence* of this **constraint**, it *need not execute the query* at all because it knows that the result of the **query will be empty**.
  - This may **save considerable time** if the **constraint checking can be done efficiently**.
  - **However**, searching through *many constraints to find those that are applicable to a given query* and that **may semantically optimize it can also be quite time-consuming**.
- With the *inclusion of active rules* in database systems *semantic query optimization techniques* may *eventually be fully incorporated* into the **DBMSs of the future**.

---

## CHAPTER – TWO: TRANSACTION PROCESSING

---

### Transaction Processing

#### 2.1 Introduction, Transaction Processing and System Concepts

---

1.

#### 2.2 Desirable Properties of Transactions

---

□ .

#### 2.3 Schedules Based on Recoverability and Serializability

---

•

#### 2.4 Transaction Support in SQL

---

## CHAPTER – THREE: CONCURRENCY CONTROL TECHNIQUES

---

### Concurrency Control Techniques

#### 3.1 Introduction, Transaction Processing and System Concepts

---

#### 3.2 Desirable Properties of Transactions

---

□ .

#### 3.3 Schedules Based on Recoverability and Serializability

---

•

#### 3.4 Transaction Support in SQL

---

:

---

## CHAPTER – FOUR: TRANSACTION PROCESSING

---

### Transaction Processing

#### 4.1 Locking Techniques for Concurrency Control

---

2.

#### 4.2 Concurrency Control Based on Timestamp Ordering

---

□ .

#### 4.3 Multiversion Concurrency Control Techniques

---

•

#### 4.4 Validation (Optimistic) Concurrency Control Techniques

---

– :

#### 4.5 Granularity of Data Items and Multiple Granularity Locking

---

•

#### 4.6 Using Locks for Concurrency Control in Indexes

---

•

---

## CHAPTER – FIVE: DISTRIBUTED DATABASE SYSTEM

### Distributed Database System

---

#### 5.1 Distributed Database Concepts, Data Fragmentation, Replication

---

.

#### 5.2 Allocation Techniques for Distributed Database Design

---

;

#### 5.3 Concurrency Control and Recovery in Distributed Databases

---

"

## **5.4 Transaction Management in Distributed Databases**

---

## **5.5 Types of Distributed Databases and Distributed DB in Oracle**

---